

KENNEDY MWANZIA

Machine Learning & Software Engineer · Retrieval, Ranking, Evaluation & Distributed Systems

Nairobi, Kenya · +254 796 516 595 · kennedymwanzia001@gmail.com

PROFESSIONAL SUMMARY

Machine learning and software engineer. I build the parts of an ML system that are usually bought: an HNSW vector index written from scratch, a two-tower retrieval model trained with sampled softmax, a point-in-time feature store, a calibrated gradient-boosted ranker, Thompson sampling for exploration, a sequential test that stops early without inflating alpha, and drift monitoring that gates a rollout. I serve them behind FastAPI at a measured p95 of 10.7 ms, put React and Next.js in front of them, and back everything with deterministic tests: **1,500+ across seven platforms**, 208 of them on a ranking system whose source type-checks clean under `mypy --strict`. I also author benchmark tasks for frontier-model evaluation, each one hardened against the same kind of shortcut before it ships.

EXPERIENCE

Software Engineer & Software Instructor

Aug 2025 – Present

Techsavanna Company · Nairobi

- **Take ML and AI features to production.** Retrieval and ranking behind stable service boundaries, model outputs validated rather than trusted, calibration so that a score means a probability, explicit confidence handling and human review as the fallback path.
- **Own the model lifecycle, not only the model.** Offline evaluation against held-out data, staged promotion through shadow and canary to live, drift monitoring on live features, and a rollback that is one call rather than a redeployment.
- **Built Aegis, a self-governing HR automation platform, end to end.** Supervised attrition modelling in scikit-learn behind a standardised pipeline that never sees absolute salary; a determinism probe, PSI and a two-sample Kolmogorov–Smirnov test combined into a PASS/WARN/BLOCK deployment gate that weights self-contradiction above input movement and verifies an LLM, a credit model or a pricing engine unmodified; and adverse-impact testing by the four-fifths rule with a chi-squared test that returns INSUFFICIENT_DATA rather than a pass when a group is too small to be evidence. **370 tests, every source module clean under mypy --strict**, on a hash-chained audit ledger, with irreversible actions gated structurally so no tenant configuration can reach them.
- **Deliver full stack, from interface to data store.** React and Next.js front ends against Java, Node and Python services, covering component and state design, API contracts, relational modelling and the integration between the layers rather than one side of it.
- **Build multi-tenant SaaS on an architecture that scales:** tenant isolation and authorisation enforced at the framework layer, service decomposition with database-per-service ownership, asynchronous messaging between contexts, idempotent write paths and defined retry and failure behaviour instead of best-effort calls.
- **Lead architecture and code review** with written technical rationale covering correctness, production cost and the recommended alternative, and deliver full-stack engineering training across Java/Spring Boot, React, Node.js and Laravel.

Benchmark Task Author, Frontier Bench / Kepler

2026 – Present

AfterQuery · Remote, contract

- Author self-contained coding and reasoning benchmark bundles for frontier-model evaluation: a Dockerised environment, sealed reference data the model under test never sees, and a root-authoritative grader that runs independently of whatever the model produces.
- **Every bundle is attacked before it ships.** An oracle solution has to clear the grader and a set of deliberate shortcut and bypass attempts has to fail it, so a bundle that cannot tell the two apart does not go out. 18 bundles built to that bar; 14 retired at my own review rather than shipped under it — too textbook, insufficient margin between a real and a shortcut solution, or, once, a verifier-isolation bypass the grader itself was vulnerable to.

SELECTED ENGINEERING PROJECTS

Sifa: Retrieval, Ranking and Experimentation Platform

2026 – Present

Python 3.12 · NumPy · SciPy · scikit-learn · FastAPI · Pydantic · pytest · mypy (strict) · Next.js · Tailwind CSS · Docker

A personalised feed built end to end without a vector database, a feature platform or a managed experiment service. The point of the project is the parts that are usually bought: the approximate nearest neighbour index, the point-in-time feature store, the calibrated ranker, the sequential test and the rollout guard are each implemented here and measured.

- **HNSW vector index written from scratch:** layered graph, greedy descent and the neighbour selection heuristic. Recall at 10 measured against exhaustive search over the same corpus, 0.820 at ef_search 32 rising to 1.000 at 256.
- **Two-tower retrieval trained with sampled softmax** over in-batch negatives, loss falling from 2.199 to 0.336 across 12 epochs, feeding a **gradient-boosted ranker at 0.973 held-out AUC with Platt scaling** fitted on a held-out slice, so a score is a probability rather than an ordering.
- **Point-in-time correct feature store** that refuses to read a value recorded after the request it is serving, which is the difference between an offline number that survives production and one that does not.
- **Exploration and experimentation:** Thompson sampling over topics with decay, hash-bucketed assignment, and a **mixture SPRT that stops early without inflating alpha**, asserted over 300 simulated A/A runs with repeated peeking.
- **Monitoring and staged rollout:** PSI and KS drift on live features, a rollout guard on click-through, calibration and latency, and a registry enforcing draft to shadow to canary to live with one-call rollback. Served at **p50 7.5 ms, p95 10.7 ms, p99 12.6 ms** over 400 requests, on 208 tests, with every source module clean under mypy strict.
- **Reports where it loses.** At 16,000 vectors the pure-Python graph is slower than a single BLAS matmul; the crossover is measured above 50,000 vectors and stated rather than hidden behind a recall figure.

SmartSeason: Agricultural Operations Platform

May – Aug 2026

Java 21 · Spring Boot 3.5 · Spring Cloud Gateway · PostgreSQL · Apache Kafka · Redis · Next.js 16 · Docker

- **27 bounded contexts across 1,650 Java sources**, database-per-service behind a gateway doing JWT pre-verification and circuit breaking. **463 tests across 27 Maven build units, generated from one service catalogue** that also emits the migrations, the Kubernetes manifests and the console.
- **Authorisation generated from a single matrix.** Eight roles over 27 services; the same table emits every @PreAuthorize and the navigation, so the menu cannot offer what a service will refuse. Verified by probing the live API as all eight roles: a farm manager may read wage rates and not write them, a farmer may read the ledger and not post to it, a worker reaching another worker's task gets a not-found rather than a forbidden.
- **Tamper-evident audit that survives concurrency.** Each record is hashed with the one before it. Under load the read-then-append path produced duplicate sequence numbers — 20 concurrent appends yielding 9 distinct sequences and a chain that failed its own verification — fixed with a unique index and a per-tenant advisory lock, then re-measured at 50 concurrent appends with the chain intact.
- **Anti-fraud time capture.** Workers start and stop tasks from the field; the timestamp is taken by the owning service, never the handset, because a device clock is the first thing anyone inflating hours would change. Ownership is enforced in the service rather than the browser.
- **Money as a financial system:** double-entry ledger with idempotent replay, M-Pesa STK push and callbacks, an order saga whose compensation unwinds in reverse, and a transactional outbox closing the crash window between commit and publish.
- **Scale work, measured rather than assumed.** PgBouncer transaction pooling took 27 services down to 3 live PostgreSQL connections and roughly doubled read throughput; per-service Redis token buckets moved by a Lua script so check-and-consume cannot interleave across replicas; list endpoints no longer count every row a tenant owns to render a caption; an autoscaler and a disruption budget on all 29 deployments, with a check that fails when a budget would block node drains instead of protecting them.

SmartRE Kenya: Real-Estate Trust Platform

Jun – Aug 2026

Java 21 · Spring Boot 3 · PostgreSQL · PgBouncer · Apache Kafka · Kubernetes · Next.js 14 · Playwright · Docker

- **Nine Spring Boot microservices and 386 tests**, each with its own PostgreSQL database behind PgBouncer transaction pooling, and a Next.js front end across 30 routes with Playwright end-to-end specs.
- **Payments as a reliability-critical subsystem:** M-Pesa escrow with idempotency keys, distributed locking, reconciliation on late callbacks and an outbox sweeper. Verified end to end against Daraja: STK push accepted, callback returned over a tunnel, secret checked, raw payload persisted and the payment moved to a terminal state on Safaricom's own reason code.
- **Document intelligence with a refusal path.** Uploads are rendered and classified by a vision model against the category the seller claimed, and a mismatch sends the submission back rather than filing the wrong document. Startup checks name what is missing — an unbound callback tunnel, an empty IP allowlist — instead of failing quietly at the first payment.

Forecourt: Revenue Assurance for Car Washes

Jul – Sep 2026

TypeScript · Express · PostgreSQL · Zod · vitest · supertest · M-Pesa Daraja · Docker

- **Three independent ledgers reconciled** for demand, work and money, each witnessed by at least two sources and at least one that is not a human, so the product is the discrepancy between records rather than the records.
- Rules-driven reconciliation over a typed domain of jobs, plates and money, with Daraja ingestion and payment matching, covered by 41 test cases under vitest and supertest.

Mara: Multi-Tenant Point-of-Sale Platform

Sep 2026 – Present

Java 21 · Spring Boot 3 · PostgreSQL row-level security · Argon2id · Flyway · Docker

- **An architecture that resolves CAP rather than hiding it.** A till must keep selling through a network partition, and a tax record must still be exactly right. The terminal is therefore the authority for what happened at its own counter and the server for what it all means together: a till never asks permission to sell, it records what it did, signs it, and is believed because it can prove authorship, ordering and that nothing was removed.
- **Tenant isolation enforced by the database, not the application.** Row-level security on every table with FORCE, and both USING and WITH CHECK — visibility alone would still let one tenant insert a row belonging to another. Exactly one cross-tenant read exists, a security-definer function that takes an enrolment-code hash and never a plaintext code, so it cannot be used to enumerate.
- **Early and stated as such:** the architecture, the platform kernel (money, fiscal leases, journal verification, terminal signatures) and the first service, on **72 tests over 25 sources**. The journal verifier already distinguishes a tampered batch from a gap in the sequence, because a batch that verifies cleanly while missing a sequence number is the more interesting fraud.

Soko: Farm-Produce Dropshipping and Supplier Routing

2026 – Present

Java 21 · Spring Boot 3.5 · PostgreSQL · Next.js 16 · Docker

- **A supplier routing engine that explains its own rejections.** Every candidate offer is checked against supplier status, stock, cold-chain capability for chilled products and lead time against a perishable's remaining shelf life, with the reason recorded for each one rejected; among what is left, it picks the cheapest offer, tie-broken by supplier reliability then lead time.
- **Idempotent stock reservation under contention:** a single conditional UPDATE ... WHERE available_qty >=: quantity takes the stock, so there is no read-then-write window for two concurrent orders to both believe they reserved the same units.
- Tenant isolation enforced at every repository method, four roles (owner, operator, supplier, customer) behind bearer-token auth, and mandatory secrets with no fallback default for either the database password or the JWT signing key.
- 12 tests covering the routing engine's rejection logic and order notifications.

TECHNICAL SKILLS

Machine Learning two-tower retrieval · sampled softmax over in-batch negatives · HNSW and approximate nearest neighbour search · gradient-boosted ranking · Platt scaling and calibration · point-in-time feature stores · Thompson sampling · MMR diversification · offline and online evaluation

Statistics & Experimentation mixture SPRT and sequential testing · A/A validation · hash-bucketed assignment · PSI and Kolmogorov–Smirnov drift detection · chi-squared testing · four-fifths-rule adverse impact · determinism probing

ML Engineering scikit-learn · NumPy · SciPy · FastAPI model serving · model registry with shadow, canary and rollback · drift and calibration gating · feature engineering · threshold calibration

Languages Python · Java 21 · TypeScript · JavaScript (ES6+) · SQL · PHP

Full-Stack Next.js 16 · React 19 · Tailwind CSS · TanStack Query · Zustand · Node.js · Express.js · Spring Boot 3 · Laravel

Data & Messaging PostgreSQL · MySQL · Redis · PgBouncer · Flyway · Apache Kafka · RabbitMQ · row-level security · advisory locks · transaction pooling

Architecture microservices · database-per-service · event-driven design · transactional outbox · saga with compensation · multi-tenancy · code generation from a service catalogue · role-based access control from a single matrix · hash-chained audit · token-bucket rate limiting · horizontal autoscaling · RFC 7807

Testing & DevOps pytest · JUnit · vitest · Playwright · mypy (strict) · ruff · Docker · Kubernetes · CI/CD · AWS · Prometheus · Grafana · Loki

EDUCATION

BSc Information Technology · KCA University, Nairobi

2021 – 2025

Software engineering, systems design, database administration and cloud computing. Microservices Architecture, a professional course with hands-on practice, 2024. AWS Certified Cloud Practitioner, in progress.